LAPORAN PRAKTIKUM KEPL

TUGAS 6: PENGALIHAN LOG STDOUT, LOG TERSTRUKTUR, ENDPOINT /HEALTH, DAN HEALTHCHECK DUA ARAH DOCKER COMPOSE



Disusun oleh:

Hafidz Rizqullah Prasetya

NIM: 24/535493/SV/24243

Kelas: PL5A1

Dosen Pengampu:

Galih Damaraji, S.Kom., M.Cs.

PROGRAM STUDI SARJANA TERAPAN
PENGEMBANGAN PERANGKAT LUNAK TERDISTRIBUSI
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

Daftar Isi

Daftar Isi **2**

Tujuan dan Lingkup Praktikum **3**
1.1 Tujuan Praktikum 3
1.2 Batasan Masalah dan Repositori Kode 3

Dasar Teori **5**
2.1 Prinsip Logging pada Arsitektur Kontainer 5
2.2 Log Terstruktur dan Perlindungan Data Sensitif 5
2.3 Mekanisme Pemeriksaan Kesehatan (Healthcheck Endpoint) 6
2.4 Orkestrasi Healthcheck Dua Arah pada Docker Compose 6

Hasil dan Pembahasan **7**
3.1 Pengalihan Kanal Log Laravel ke Stdout (Soal 1) 7
3.2 Penerapan Tiga Log Terstruktur Berkonteks (Soal 2) 8
3.2.1 Alur 1: Penyimpanan Data Aset Baru (INFO) 8
3.2.2 Alur 2: Validasi Masukan Gagal (WARNING) 9
3.2.3 Alur 3: Gagal Menghubungi Basis Data (ERROR) 9
3.2.4 Potongan Log Terstruktur di Terminal 10
3.3 Endpoint Pemeriksaan Kesehatan /health (Soal 3) 10
3.3.1 Balasan Saat Sehat (HTTP 200) 11
3.3.2 Balasan Saat Basis Data Mati (HTTP 503) 11
3.4 Konfigurasi Healthcheck pada Compose (Soal 4) 12
3.5 Pengujian Healthcheck Dua Arah (Soal 5) 13
3.5.1 Sehat ke Sakit 13
3.5.2 Sakit Kembali ke Sehat 14
3.6 Pembatasan Ukuran Log (Soal 6) 15
3.7 Perubahan Kode di Repositori GitHub 15

Kendala dan Solusi **17**

Kesimpulan **18**

Daftar Pustaka **19**

Tujuan dan Lingkup Praktikum

1.1 Tujuan Praktikum

Tujuan pelaksanaan praktikum Tugas 6 mata kuliah Konstruksi dan Evolusi Perangkat Lunak ini adalah:

1. Mahasiswa mampu mengonfigurasi kanal pencatatan log pada kerangka kerja Laravel agar dialihkan langsung ke aliran standar keluaran (`stdout`) di dalam kontainer, sehingga log dapat dikelola secara terpusat oleh mesin Docker.
2. Mahasiswa mampu merancang dan menerapkan sekurang-kurangnya tiga log terstruktur dengan konteks lengkap (pelaku, aktivitas, dan alasan terjadinya kejadian) pada alur penting aplikasi tanpa mencatat data rahasia maupun informasi pribadi.
3. Mahasiswa mampu membangun endpoint inspeksi kesehatan sistem (`/health`) yang memverifikasi kesiapan operasional basis data relasional PostgreSQL dan subsistem penyimpanan *cache*.
4. Mahasiswa mampu mendefinisikan mekanisme *healthcheck* deklaratif pada berkas `compose.yaml` dengan parameter interval, batas waktu (*timeout*), jumlah percobaan ulang (*retries*), dan periode pemanasan (*start_period*).
5. Mahasiswa mampu membuktikan keandalan mekanisme pemulihan dua arah ketika layanan basis data dimatikan secara sengaja hingga kontainer berstatus *unhealthy*, serta mengamati pemulihan kembali ke status *healthy* setelah layanan dinyalakan ulang.
6. Mahasiswa mampu mengonfigurasi pembatasan ukuran berkas log kontainer menggunakan opsi `max-size` dan `max-file` pada pengaturan *logging* Docker Compose.

1.2 Batasan Masalah dan Repositori Kode

Pengerjaan tugas ini melanjutkan repositori ujian tengah semester (UTS) berbasis Docker Compose yang mengelola Sistem Inventaris Aset Laboratorium TRPL. Seluruh perubahan kode aplikasi, konfigurasi lingkungan, skrip pengujian fitur, serta berkas orkestrasi kontainer dikelola pada repositori publik GitHub:

https://github.com/hafidzrizqullahprasetya/uts-p1-535493

Layanan aplikasi terdiri dari peladen backend berbasis FrankenPHP (PHP 8.5.12 Alpine) yang menyajikan RESTful API, basis data PostgreSQL 18.6, dan antarmuka frontend Vue.js 3 berbasis peladen Nginx.

Dasar Teori

2.1 Prinsip Logging pada Arsitektur Kontainer

Metodologi *Twelve-Factor App* pada poin kesebelas menyatakan bahwa aplikasi modern tidak boleh mengelola berkas pencatatan log secara mandiri di dalam sistem berkas lokalnya. Menulis log ke berkas statis di dalam kontainer menimbulkan beberapa masalah: log hilang saat kontainer dihapus, rotasi berkas harus diatur sendiri, dan berkas log dapat terus membesar hingga memenuhi disk [1].

Sebagai gantinya, aplikasi sebaiknya mengalirkan seluruh pesan log kejadian ke aliran standar keluaran (`stdout`) atau standar galat (`stderr`) tanpa *buffering*. Docker menangkap aliran tersebut dan mengarahkannya ke *logging driver* yang telah ditentukan, seperti `json-file`, `syslog`, atau penyedia agregasi log terpusat [2].

2.2 Log Terstruktur dan Perlindungan Data Sensitif

Pencatatan log berbasis teks bebas seringkali menyulitkan proses pemfilteran, analisis tren, dan pelacakan akar masalah saat terjadi insiden. Pendekatan log terstruktur (*structured logging*) menyusun setiap entri log ke dalam format pasangan kunci dan nilai, biasanya direpresentasikan dalam JSON atau larik terindeks PSR-3 [3].

Setiap entri log wajib membawa metadata kontekstual yang menjawab tiga pertanyaan mendasar:

1. **Siapa**: Identitas entitas yang memicu kejadian (misalnya pengenal pengguna atau agen sistem).
2. **Aktivitas**: Operasi teknis atau alur bisnis yang sedang dijalankan.
3. **Sebab**: Alasan terjadinya status tersebut, baik berupa keberhasilan eksekusi maupun pesan galat dari pengecualian sistem.

Log tidak boleh memuat kata sandi, token autentikasi seperti token Sanctum, kunci sesi, maupun data pribadi seperti email, nomor telepon, atau nomor identitas [4].

2.3 Mekanisme Pemeriksaan Kesehatan (Healthcheck Endpoint)

Endpoint `/health` berfungsi sebagai indikator kesiapan operasional (*readiness probe*) dan ketahanan hidup (*liveness probe*) dari sebuah layanan web. Endpoint kesehatan perlu menjalankan pemeriksaan aktif terhadap dependensi kritis yang menopang aplikasi [5].

Apabila seluruh komponen inti, seperti koneksi basis data relasional dan pustaka penyimpanan *cache*, dapat dihubungi dan membalas perintah uji coba dengan benar, endpoint merespons dengan kode status HTTP 200 OK. Jika salah satu komponen gagal, endpoint mengembalikan kode status HTTP 503 Service Unavailable disertai laporan diagnostik yang menyebutkan secara spesifik komponen mana yang bermasalah.

2.4 Orkestrasi Healthcheck Dua Arah pada Docker Compose

Docker Compose menyediakan instrumen deklaratif untuk memantau status kesehatan kontainer secara berkala melalui blok `healthcheck`. Konfigurasi ini memiliki empat parameter utama:

1. `interval`: Frekuensi eksekusi perintah pemeriksaan (misalnya setiap 10 detik).
2. `timeout`: Batas durasi maksimal yang diberikan bagi perintah pemeriksaan sebelum dianggap gagal (misalnya 5 detik).
3. `retries`: Ambang batas kegagalan beruntun yang diperlukan sebelum Docker mengubah status kontainer dari *healthy* menjadi *unhealthy*.
4. `start_period`: Masa tenggang awal saat kontainer pertama kali dinyalakan agar proses inisialisasi aplikasi tidak langsung divonis gagal.

Pemeriksaan dua arah berarti status kontainer dapat berpindah secara dinamis: dari sehat menjadi sakit saat dependensi eksternal terputus, dan kembali sehat saat dependensi tersebut pulih tanpa perlu membuat ulang kontainer [6].

Hasil dan Pembahasan

Seluruh perintah pada bab ini dijalankan dari akar repositori `uts-p1-535493`. Repositori tersebut masih menyimpan berkas `docker-compose.yml` lama dari UTS di samping `compose.yaml`, sehingga sebelum pengujian terminal diatur dengan `export COMPOSE_FILE=compose.yaml` agar Docker Compose hanya membaca `compose.yaml`.

3.1 Pengalihan Kanal Log Laravel ke Stdout (Soal 1)

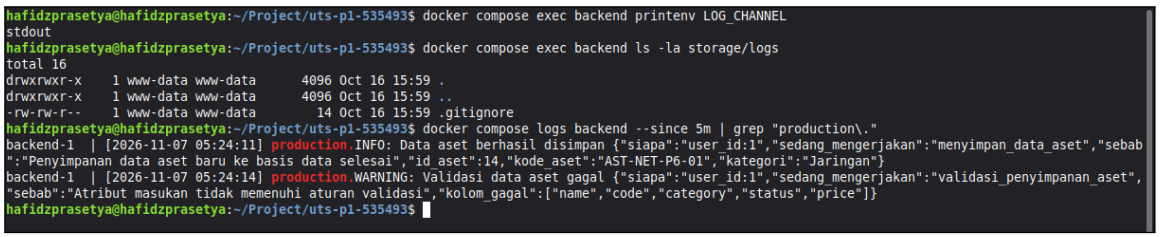
Secara bawaan Laravel memakai kanal `stack` yang menulis ke berkas `storage/logs/laravel.log` di dalam kontainer. Pada berkas `backend/config/logging.php` ditambahkan kanal `stdout` yang memakai `StreamHandler` dengan tujuan `php://stdout`, seperti pada Listing 1.

```
'stdout' => [
    'driver' => 'monolog',
    'level' => env('LOG_LEVEL', 'debug'),
    'handler' => StreamHandler::class,
    'handler_with' => [
        'stream' => 'php://stdout',
    ],
    'formatter' => env('LOG_STDOUT_FORMATTER'),
    'processors' => [PsrLogMessageProcessor::class],
],
```

Listing 1: Penambahan channel stdout pada backend/config/logging.php.

Variabel `LOG_CHANNEL` pada layanan `backend` di `compose.yaml` dan pada `backend/.env.example` diubah menjadi `stdout`. Docker menangkap semua yang ditulis proses ke `stdout`, sehingga log aplikasi dapat dibaca dengan `docker compose logs`.

Gambar 1 memperlihatkan tiga pemeriksaan. Perintah `printenv LOG_CHANNEL` di dalam kontainer menghasilkan `stdout`. Direktori `storage/logs` hanya berisi `.gitignore`, artinya tidak ada berkas `laravel.log` yang dibuat. Terakhir, `docker compose logs backend` menampilkan baris log aplikasi `production.INFO` dan `production.WARNING`.



Gambar 1: Kanal log bernilai `stdout`, tidak ada berkas log di `storage/logs`, dan log aplikasi terbaca lewat `docker compose logs`.

3.2 Penerapan Tiga Log Terstruktur Berkonteks (Soal 2)

Log ditulis dengan facade `Illuminate\Support\Facades\Log`. Pesan utama dibuat singkat, sedangkan detailnya diletakkan pada larik konteks yang dicetak sebagai JSON. Setiap log memuat tiga kunci yang sama:

- `siapa`: pelaku kejadian, berupa `user_id:<id>` untuk pengguna atau `sistem_pemantau` untuk pemeriksaan kesehatan.
- `sedang_mengerjakan`: operasi yang sedang berjalan.
- `sebab`: alasan kejadian, baik keterangan keberhasilan maupun pesan pengecualian.

Pelaku dicatat dengan nomor `id` pengguna, bukan email. Email termasuk data pribadi, sedangkan `id` cukup untuk menelusuri pengguna di basis data bila diperlukan. Password, token Sanctum, dan isi masukan formulir tidak pernah dimasukkan ke konteks log.

3.2.1 Alur 1: Penyimpanan Data Aset Baru (INFO)

Alasan memilih alur ini: menambah aset adalah operasi tulis utama pada sistem inventaris. Bila ada selisih data, log ini menjadi jejak kapan aset dibuat, oleh pengguna mana, dan dengan kode apa. Kejadian ini normal sehingga tingkat yang dipakai adalah `info`.

```php
Log::info('Data aset berhasil disimpan', [
    'siapa' => 'user_id:'.($request->user()?->id ?? 'tidak_diketahui'),
    'sedang_mengerjakan' => 'menyimpan_data_aset',
    'sebab' => 'Penyimpanan data aset baru ke basis data selesai',
    'id_aset' => $asset->id,
    'kode_aset' => $asset->code,
    'kategori' => $asset->category,
]);
```

Listing 2: Log info pada AssetController::store() setelah aset tersimpan.

3.2.2 Alur 2: Validasi Masukan Gagal (WARNING)

Alasan memilih alur ini: permintaan yang ditolak validasi tidak merusak sistem, tetapi bila sering terjadi bisa menandakan formulir frontend yang salah atau klien yang mengirim data asal. Tingkat `warning` dipakai karena kejadian ini perlu diperhatikan tetapi bukan kegagalan sistem. Konteks hanya menyimpan nama kolom yang gagal (`kolom_gagal`), bukan isi masukan pengguna.

```php
if ($validator->fails()) {
    Log::warning('Validasi data aset gagal', [
        'siapa' => 'user_id:'.($request->user()?->id ?? 'tidak_diketahui'),
        'sedang_mengerjakan' => 'validasi_penyimpanan_aset',
        'sebab' => 'Atribut masukan tidak memenuhi aturan validasi',
        'kolom_gagal' => array_keys($validator->errors()->toArray()),
    ]);
    return response()->json([
        'message' => 'Validasi gagal',
        'errors' => $validator->errors(),
    ], 422);
}
```

Listing 3: Log warning saat validasi penyimpanan aset gagal.

3.2.3 Alur 3: Gagal Menghubungi Basis Data (ERROR)

Alasan memilih alur ini: basis data adalah dependensi yang paling sering membuat aplikasi tidak bisa melayani permintaan. Saat koneksi putus, pengembang perlu tahu sebab teknisnya, misalnya host `db` tidak dapat di-*resolve* atau server masih dalam proses menyala. Tingkat `error` dipakai karena layanan benar-benar gagal. Log ini dipasang di dua tempat: blok `catch` pada `HealthController` (Listing 4) dan blok `catch` pada metode `index()`, `store()`, serta `update()` di `AssetController` (Listing 5).

```php
} catch (\Throwable $e) {
    // ... status database ditandai unhealthy
    Log::error('Healthcheck mendeteksi kegagalan koneksi basis data', [
        'siapa' => 'sistem_pemantau',
        'sedang_mengerjakan' => 'pemeriksaan_kesehatan_database',
        'sebab' => $e->getMessage(),
    ]);
}
```

Listing 4: Log error pada HealthController saat basis data tidak dapat dihubungi.

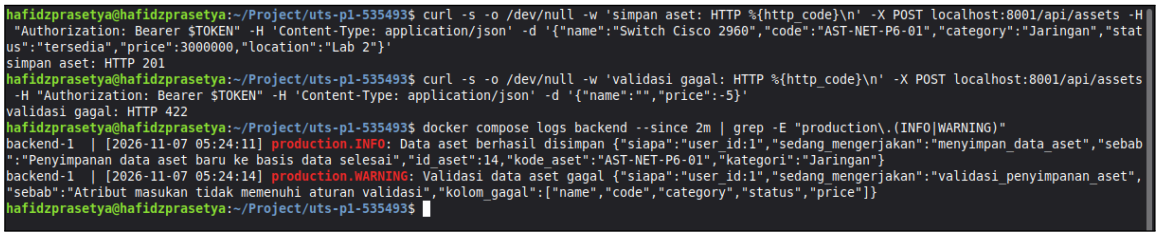
```php
} catch (\Throwable $e) {
    Log::error('Gagal menghubungi basis data saat menyimpan aset', [
        'siapa' => 'user_id:'.($request->user()?->id ?? 'tidak_diketahui'),
        'sedang_mengerjakan' => 'menyimpan_data_aset',
```

```
        'sebab' => $e->getMessage(),
    ]);
    // ... balas HTTP 500 ke klien
}
```

Listing 5: Log error pada AssetController::store() saat penyimpanan gagal karena basis data.

3.2.4 Potongan Log Terstruktur di Terminal

Untuk memunculkan log INFO dan WARNING, token login disimpan ke variabel `$TOKEN` tanpa dicetak ke layar. Setelah itu dikirim satu permintaan tambah aset yang valid (HTTP 201) dan satu permintaan dengan nama kosong serta harga negatif (HTTP 422). Gambar 2 memperlihatkan kedua log yang dihasilkan. Kolom `siapa` berisi `user_id:1`, dan tidak ada email, password, maupun token di dalamnya. Log tingkat ERROR muncul saat basis data dimatikan dan ditampilkan pada Gambar 7 di Soal 5.



Gambar 2: Permintaan simpan aset (201) dan validasi gagal (422) beserta log INFO dan WARNING yang dihasilkan.

Untuk menjaga aturan ini tetap berlaku pada perubahan berikutnya, ditambahkan pengujian `test_asset_logs_do_not_contain_personal_data_or_token` pada `AssetApiTest`. Pengujian ini memakai `Log::spy()` dan memastikan konteks log info dan warning berisi `user_id`, serta tidak memuat email, token, ataupun kata `password`.

3.3 Endpoint Pemeriksaan Kesehatan /health (Soal 3)

Endpoint `/health` ditangani oleh `HealthController::check()` dan melakukan dua pemeriksaan nyata:

1. **Basis data:** memanggil `DB::connection()->getPdo()` lalu menjalankan `SELECT 1` ke PostgreSQL.
2. **Cache:** menulis kunci acak dengan `Cache::put()`, membacanya kembali, membandingkan nilainya, lalu menghapusnya.

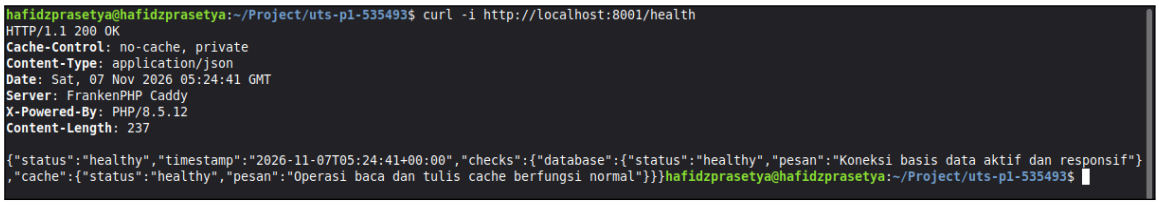
Jika kedua pemeriksaan lulus, endpoint membalas HTTP 200. Jika salah satu gagal, endpoint membalas HTTP 503 dan menambahkan larik `bagian_bermasalah` yang menyebut

komponen yang gagal. Rute didaftarkan melalui parameter `then` pada `withRouting()` di `backend/bootstrap/app.php`, sehingga tersedia di `/health` tanpa awalan `/api` dan tanpa middleware sesi.

Perilaku ini juga diuji otomatis di `HealthCheckTest`: kondisi sehat (200), basis data gagal (503 dengan `bagian_bermasalah: ["database"]`), dan cache gagal (503 dengan `bagian_bermasalah: ["cache"]`).

3.3.1 Balasan Saat Sehat (HTTP 200)

Gambar 3 memperlihatkan hasil `curl -i http://localhost:8001/health` saat semua layanan menyala.



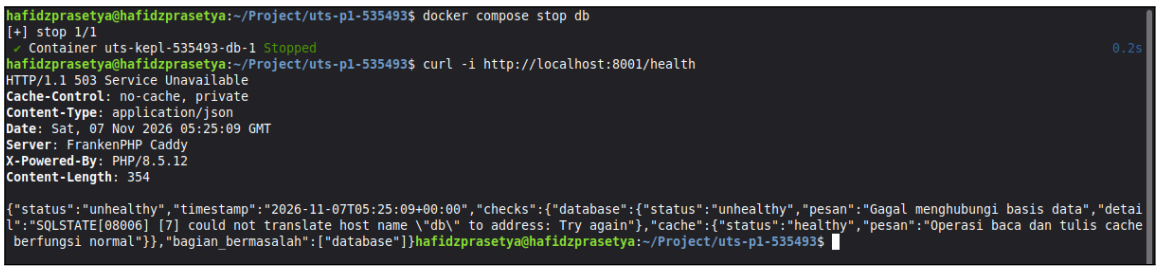
Gambar 3: Balasan `/health` dengan status HTTP 200 OK saat basis data dan cache sehat.

```
{
  "status": "healthy",
  "timestamp": "2026-11-07T05:24:41+00:00",
  "checks": {
    "database": {
      "status": "healthy",
      "pesan": "Koneksi basis data aktif dan responsif"
    },
    "cache": {
      "status": "healthy",
      "pesan": "Operasi baca dan tulis cache berfungsi normal"
    }
  }
}
```

Listing 6: Isi balasan /health saat sehat.

3.3.2 Balasan Saat Basis Data Mati (HTTP 503)

Setelah layanan `db` dihentikan dengan `docker compose stop db`, permintaan yang sama dibalas HTTP 503 Service Unavailable (Gambar 4). Komponen `database` berstatus *unhealthy* dengan sebab `could not translate host name "db"`, sedangkan `cache` tetap *healthy* karena memakai driver `file`.



Gambar 4: Balasan `/health` dengan status HTTP 503 setelah layanan `db` dihentikan.

```
{
  "status": "unhealthy",
  "timestamp": "2026-11-07T05:25:09+00:00",
  "checks": {
    "database": {
      "status": "unhealthy",
      "pesan": "Gagal menghubungi basis data",
      "detail": "SQLSTATE[08006] [7] could not translate host name \"db\" to address
↪ : Try again"
    },
    "cache": {
      "status": "healthy",
      "pesan": "Operasi baca dan tulis cache berfungsi normal"
    }
  },
  "bagian_bermasalah": ["database"]
}
```

Listing 7: Isi balasan /health saat basis data mati.

3.4 Konfigurasi Healthcheck pada Compose (Soal 4)

Healthcheck layanan `backend` di `compose.yaml` memanggil `curl -fs` ke `/health` dari dalam kontainer. Opsi `-f` membuat `curl` keluar dengan kode gagal bila server membalas 4xx atau 5xx, sehingga balasan 503 terhitung sebagai pemeriksaan gagal. Parameter yang dipakai:

- `interval: 10s`: pemeriksaan dijalankan setiap 10 detik.
- `timeout: 5s`: pemeriksaan dianggap gagal bila tidak selesai dalam 5 detik.
- `retries: 5`: status menjadi *unhealthy* setelah 5 kali gagal berturut-turut.
- `start_period: 15s`: kegagalan selama 15 detik pertama setelah kontainer menyala tidak dihitung.

```yaml
backend:
  build: ./backend
  restart: unless-stopped
  environment:
    APP_ENV: production
    APP_DEBUG: "false"
    LOG_CHANNEL: stdout
    CACHE_STORE: file
    DB_CONNECTION: pgsql
    DB_HOST: db
    # ... variabel lain diambil dari .env
  depends_on:
    db:
      condition: service_healthy
  ports:
    - "${BACKEND_PORT:-8001}:8000"
  healthcheck:
    test: ["CMD-SHELL", "curl -fs http://localhost:8000/health || exit 1"]
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 15s
  logging:
    driver: "json-file"
    options:
      max-size: "10m"
      max-file: "3"
```

Listing 8: Konfigurasi layanan backend pada compose.yaml.

Layanan db juga memiliki healthcheck `pg_isready`, dan `backend` baru dijalankan setelah db sehat melalui `depends_on: condition: service_healthy`. Gambar 5 memperlihatkan kolom STATUS pada `docker compose ps` bernilai `Up (healthy)` untuk `backend` dan db. Layanan `frontend` tidak diberi healthcheck sehingga hanya tertulis `Up`.

```
hafidzprasetya@hafidzprasetya:~/Project/uts-p1-535493$ docker compose ps
NAME                      IMAGE                     COMMAND                  SERVICE    CREATED          STATUS                    PORTS
uts-kepl-535493-backend-1    uts-kepl-535493-backend    "docker-php-entrypoi…"   backend    2 minutes ago    Up 2 minutes (healthy)    80/tcp, 443/tcp,
2019/tcp, 443/udp, 0.0.0.0:8001->8000/tcp, [::]:8001->8000/tcp
uts-kepl-535493-db-1         postgres:18.6-alpine      "docker-entrypoint.s…"   db         12 hours ago     Up 12 hours (healthy)     5432/tcp
uts-kepl-535493-frontend-1   uts-kepl-535493-frontend   "/docker-entrypoint.…"   frontend   12 hours ago     Up 12 hours               0.0.0.0:8080->80/
tcp, [::]:8080->80/tcp
hafidzprasetya@hafidzprasetya:~/Project/uts-p1-535493$
```

Gambar 5: Keluaran `docker compose ps` saat `backend` dan db berstatus *healthy*.

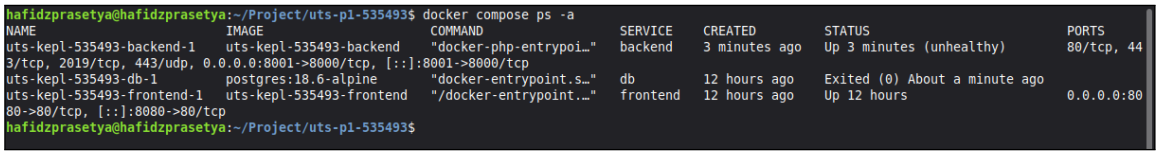
3.5 Pengujian Healthcheck Dua Arah (Soal 5)

3.5.1 Sehat ke Sakit

Layanan basis data dimatikan dengan sengaja:

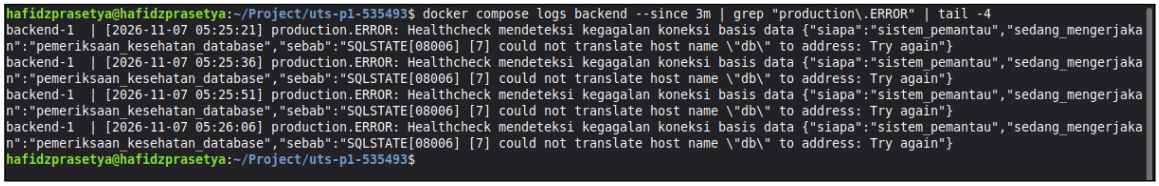
```
docker compose stop db
```

Sejak saat itu setiap pemeriksaan `/health` dibalas 503, sehingga `curl -fs` gagal. Docker tidak langsung mengubah status; status baru menjadi *unhealthy* setelah 5 pemeriksaan gagal berturut-turut. Pada pengujian ini `db` dihentikan sekitar pukul 12.25 WIB, dan `docker inspect` mencatat status *unhealthy* dengan `FailingStreak` 5 pada pukul 12.26.03 WIB. Gambar 6 memperlihatkan `docker compose ps -a` pada saat itu: `backend` berstatus `Up (unhealthy)`, sedangkan `db` berstatus `Exited (0)`.



Gambar 6: Keluaran `docker compose ps -a` setelah `db` dihentikan: `backend` berstatus *unhealthy*.

Sebab perubahan status tersebut tercatat di log. Gambar 7 memperlihatkan log `production.ERROR` dari `HealthController` yang muncul kira-kira setiap 15 detik selama `db` mati. Setiap baris menyebut pelaku (`sistem_pemantau`), kegiatan (`pemeriksaan_kesehatan_database`), dan sebab teknisnya (`could not translate host name "db"`).



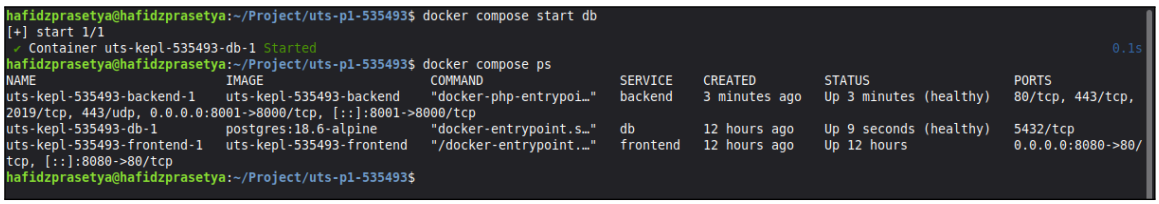
Gambar 7: Log ERROR yang mencatat sebab kegagalan healthcheck selama basis data mati.

3.5.2 Sakit Kembali ke Sehat

Layanan basis data dinyalakan lagi:

```
docker compose start db
```

Setelah PostgreSQL siap menerima koneksi, pemeriksaan `/health` berikutnya kembali dibalas 200 dan status `backend` berubah menjadi *healthy*. Pada pengujian ini status *healthy* tercatat pada pukul 12.26.26 WIB, kurang dari setengah menit setelah `db` dinyalakan. Gambar 8 memperlihatkan `backend` kembali `Up (healthy)` dan `db` berstatus `Up 9 seconds (healthy)`. Kolom CREATED pada `backend` tetap “3 minutes ago”, yang menunjukkan kontainer backend tidak dibuat ulang ataupun di-*restart*.

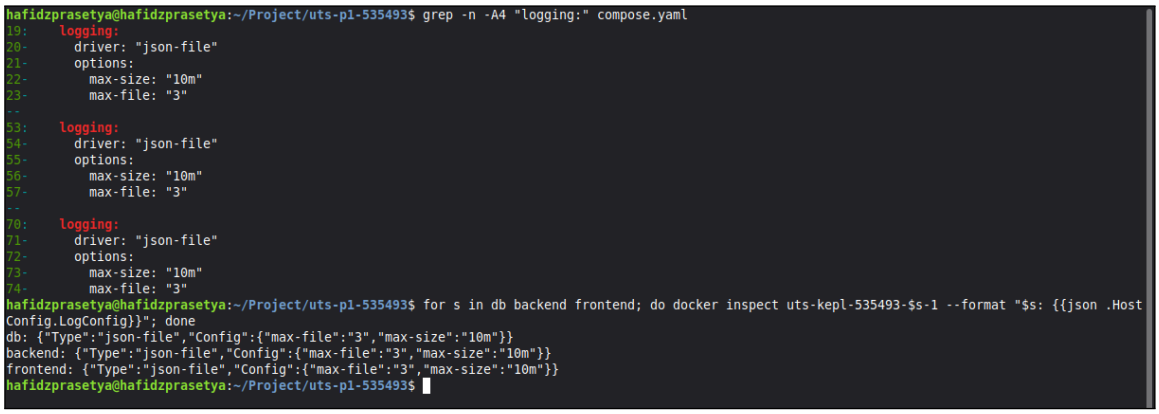


Gambar 8: Setelah `docker compose start db`, `backend` kembali berstatus *healthy*.

3.6 Pembatasan Ukuran Log (Soal 6)

Driver `json-file` bawaan Docker menyimpan keluaran `stdout` kontainer ke berkas di host. Tanpa batas, berkas ini terus membesar. Karena itu setiap layanan di `compose.yaml` (`db`, `backend`, dan `frontend`) diberi blok `logging` dengan `max-size: "10m"` dan `max-file: "3"`. Docker merotasi berkas log saat ukurannya mencapai 10 MB dan menyimpan paling banyak 3 berkas, sehingga log satu kontainer tidak melebihi sekitar 30 MB [2].

Gambar 9 memperlihatkan ketiga blok `logging` di `compose.yaml` dan hasil `docker inspect` yang menunjukkan konfigurasi tersebut aktif pada ketiga kontainer yang sedang berjalan.



Gambar 9: Blok `logging` di `compose.yaml` dan konfigurasi log yang aktif pada tiap kontainer.

3.7 Perubahan Kode di Repositori GitHub

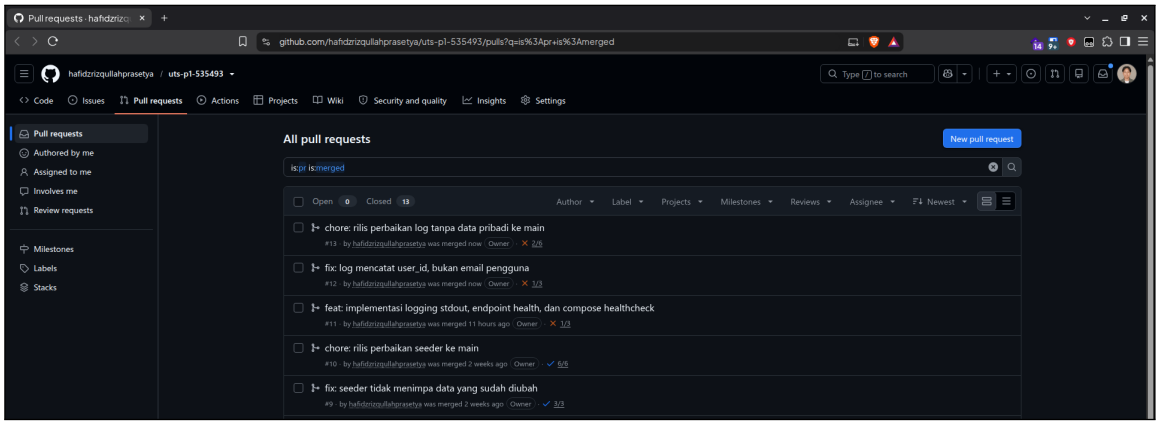
Perubahan dimasukkan melalui Pull Request di repositori https://github.com/hafidzrizqullahprasetya/uts-p1-535493:

1. PR #11 menambahkan kanal `stdout`, log terstruktur, `HealthController`, `HealthCheckTest`, serta healthcheck dan pembatasan log di `compose.yaml`.

2. PR #12 (`fix/task-6-log-tanpa-data-pribadi` ke `dev`) mengganti email pada kolom `siapa` menjadi `user_id`, menyeragamkan nama kunci konteks, dan menambah pengujian log tanpa data pribadi.

3. PR #13 (`dev` ke `main`) membawa perbaikan tersebut ke cabang utama.

Gambar 10 memperlihatkan ketiga PR tersebut dalam status *merged*. Tanda silang pada PR tersebut berasal dari GitHub Actions yang tidak berjalan, bukan dari pengujian yang gagal (lihat bab Kendala dan Solusi).



Gambar 10: Daftar Pull Request yang sudah di-*merge* pada repositori `uts-p1-535493`.

Kendala dan Solusi

1. **Log sempat mencatat email pengguna.** Versi pertama log aset mengisi kolom `siapa` dengan email. Soal melarang pencatatan data pribadi, jadi kolom tersebut diganti `user_id:<id>` dan ditambahkan pengujian yang gagal bila email, token, atau kata `password` muncul di konteks log.

2. **Status *unhealthy* tidak langsung muncul.** Tangkapan `docker compose ps` yang diambil sesaat setelah `db` dihentikan masih menampilkan `backend` sebagai *healthy*, karena Docker menunggu 5 kegagalan beruntun dengan jeda 10 detik. Solusinya adalah memantau `docker inspect` sampai `FailingStreak` mencapai 5 dan status berubah menjadi *unhealthy*, baru kemudian mengambil tangkapan layar.

3. **Permintaan `/health` lambat saat `db` mati.** Setelah kontainer `db` berhenti, pencarian nama host `db` di DNS internal Docker memakan beberapa detik sebelum gagal, sehingga `curl` menunggu sejenak sebelum menerima 503. Hasil pengujian tidak berubah karena Docker menghitung balasan 503 maupun *timeout* sebagai kegagalan. Jeda ini juga membuat log ERROR muncul kira-kira setiap 15 detik, bukan tepat 10 detik.

4. **Peringatan dua berkas compose.** Repositori menyimpan `compose.yaml` dan `docker-compose.yml` (berkas UTS yang dirujuk laporan UTS), sehingga Docker Compose menampilkan peringatan `Found multiple config files`. Berkas lama tidak dihapus agar rujukan laporan UTS tetap berlaku; sebagai gantinya pengujian memakai `export COMPOSE_FILE=compose.yaml`.

5. **GitHub Actions tidak berjalan.** Job CI pada PR #11 sampai #13 berhenti dalam 2 detik dengan pesan *"The job was not started because your account is locked due to a billing issue"*. Karena proteksi cabang mewajibkan status `Backend Test` dan `Frontend Build` lulus, pengujian dijalankan secara lokal (`php artisan test`: 17 lulus, 59 asersi). Opsi *enforce admins* pada cabang `dev` dan `main` dimatikan sementara untuk melakukan merge PR #12 dan #13 sebagai admin, lalu diaktifkan kembali. CI perlu dijalankan ulang setelah masalah billing akun selesai.

Kesimpulan

1. Log Laravel dialihkan ke `stdout` lewat kanal `stdout` dan `LOG_CHANNEL=stdout`. Tidak ada berkas `laravel.log` di dalam kontainer, dan log aplikasi terbaca melalui `docker compose logs`.

2. Tiga alur penting diberi log terstruktur dengan tingkat yang sesuai: penyimpanan aset (`info`), validasi gagal (`warning`), dan gagal menghubungi basis data (`error`). Setiap log memuat `siapa`, `sedang_mengerjakan`, dan `sebab`. Pelaku dicatat sebagai `user_id`, dan pengujian otomatis memeriksa bahwa email, token, dan password tidak tercatat.

3. Endpoint `/health` memeriksa basis data dengan `SELECT 1` dan cache dengan operasi tulis-baca. Endpoint membalas 200 saat sehat dan 503 dengan `bagian_bermasalah` saat ada komponen yang gagal.

4. Healthcheck `backend` di `compose.yaml` memakai `interval` 10 detik, `timeout` 5 detik, `retries` 5, dan `start_period` 15 detik, dan kolom STATUS pada `docker compose ps` menampilkan *healthy*.

5. Saat `db` dihentikan, `backend` menjadi *unhealthy* setelah sekitar satu menit dan log ERROR mencatat sebabnya. Setelah `db` dinyalakan kembali, `backend` kembali *healthy* tanpa dibuat ulang.

6. Setiap layanan memakai driver `json-file` dengan `max-size: "10m"` dan `max-file: "3"`, dan konfigurasi ini terlihat aktif melalui `docker inspect`.

Daftar Pustaka

[1] A. Wiggins, "The Twelve-Factor App: XI. Logs - Treat logs as event streams," 2017. [Daring]. Tersedia: https://12factor.net/logs. [Diakses: 06-Nov-2026].

[2] Docker Inc., "Configure logging drivers - JSON File logging driver," Docker Documentation, 2026. [Daring]. Tersedia: https://docs.docker.com/engine/logging/drivers/json-file/. [Diakses: 06-Nov-2026].

[3] J. Selda, "Monolog - Logging for PHP 8," GitHub Repository, 2026. [Daring]. Tersedia: https://github.com/Seldaek/monolog. [Diakses: 06-Nov-2026].

[4] OWASP Foundation, "Logging Cheat Sheet - Sensitive Data Protection and Storage," OWASP Cheat Sheet Series, 2026. [Daring]. Tersedia: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html. [Diakses: 06-Nov-2026].

[5] Docker Inc., "Dockerfile reference: HEALTHCHECK instruction," Docker Documentation, 2026. [Daring]. Tersedia: https://docs.docker.com/reference/dockerfile/#healthcheck. [Diakses: 06-Nov-2026].

[6] Compose Open Source Community, "Compose file specification: healthcheck attribute," 2026. [Daring]. Tersedia: https://docs.docker.com/compose/compose-file/05-services/#healthcheck. [Diakses: 06-Nov-2026].